EL-2003 – Computer Organization & Assembly Language Lab
Lab # 9

Objectives:

- Use shifts, and, xor, test, rotates and the carry flag to work on individual bits.
- Choose the right jump (signed or unsigned) for the data.
- Write subroutines that take parameters in registers, in flags, or on the stack.
- Save and restore registers around a subroutine.
- Call one subroutine from another.
- Clean the stack correctly.

Note: Carefully read the following instructions (*Each instruction contains a weightage*)

1. First think about statement problems and then write your program.
2. Do not copy from any source otherwise you will be penalized with negative marks.
3. Add your source code in word document and attach output screenshots.
4. Upload word file with codes and screenshot.
5. Submit your lab on Google Classroom.

Question 1:

A number is a power of two (1, 2, 4, 8, and so on) when exactly one of its bits is set. Write is_pow2, which takes an unsigned word in AX and reports the answer in the carry flag: CF=1 if it is a power of two, CF=0 if not. Zero is not a power of two. It must decide using only and, sub and a conditional jump, and AX must not change. Write bitpos, which takes a power of two in AX and returns the position of its set bit in AX (so 1 gives 0, 2 gives 1, 8 gives 3), using shifts. Write classify, which takes a word in AX, calls is_pow2, and if the answer is yes calls bitpos and finishes with CF=1 and the position in AX. If the answer is no it finishes with CF=0 and AX unchanged. Run classify from a loop in the main code over dw 1, 6, 64, 100, 1024, 0, 4096, 96. Count the powers of two in pow_count and add up their positions in exp_sum.

Question 2:

The array arr is dw 9, -3, 14, 0, -8, 5, 22, -1 with 8 signed words. Write swap_if_greater, which takes the address of a word in BX and compares the signed word at [bx] with the signed word at [bx+2]. If the first is greater it swaps the two words in memory using only push and pop (no spare register for the swap) and reports CF=1. Otherwise it changes nothing and reports CF=0. It must give back every register. In the main code, sort the array into ascending signed order with a bubble sort: 7 passes, and in each pass 7 neighbouring comparisons starting from the first element, always calling swap_if_greater. Count the number of swaps made in total in swaps.

Question 3:

Write add32, which takes two 32-bit numbers on the stack. The first is pushed as low word then high word, and the second the same way, so four pushes in all. It returns the 32-bit sum in DX:AX. It reaches its parameters through BP and removes them with ret 8. The data are three 32-bit numbers stored as low-word, high-word pairs: dw 0x1170,0x0001, 0xFBD0,0x0001, 0xD090,0x0003. Starting from a total of zero (stored in total_lo and total_hi), add all three using add32, keeping the running total in those two variables. Then compare the final total with the limit stored as limit_lo: dw 0x1A80 and limit_hi: dw 0x0006, using unsigned jumps, comparing the high words first. Set over to 1 if the total is greater than the limit and leave it 0 otherwise. Finally, divide the total by 2 with a 32-bit right shift and store the result in half_lo and half_hi.

Question 4:

A hall has 16 seats, numbered 0 to 15, and a single word bitmap (starting at 0) records them: bit N set means seat N is booked. Requests arrive as pairs of words, an operation then a seat number, where operation 1 means book and operation 2 means cancel: dw 1,3, 1,7, 1,3, 1,12, 2,7, 2,9, 1,15. Write make_mask, which takes a seat number in AX and returns a word in AX with only that seat's bit set. Write is_booked, which takes a seat number on the stack, reports CF=1 if that seat is booked and CF=0 if not, and removes its parameter with ret 2. Write book_seat and cancel_seat, which each take a seat number on the stack, remove it with ret 2, and report CF=0 if the request succeeded or CF=1 if it was rejected. Booking an already-booked seat is rejected, and cancelling a seat that is not booked is rejected. Both of them must call is_booked, and is_booked must call make_mask. They keep booked_count up to date. In the main code, process all 7 requests in order and count the rejected ones in rejected.